Remove Namespace relops From C++26

Document #:	P3047R0
Date:	2024-02-15
Project:	Programming Language C++
Audience:	Library Evolution

Reply-to:	Alisdair Meredith <ameredith1@bloomberg.net>

Contents

§ 1 Abstract

Namespace `std::relops` was provided by the original C++ Standard, comprising a set of comparison operator function templates that can provide a full set of comparison operators for a type that supports a minimal subset of those operators. That namespace was deprecated by C++20 as the operator rewrite rules provide similar functionality directly in the language, and this paper proposes removing that namespace from C++26.

§ 2 Revision history

§ R0 January 2023 (midterm mailing)

— Initial draft of this paper, without feedback from Kona 2023

§ 3 Introduction

At the start of the C++23 cycle, [P2139R2] tried to review each deprecated feature of C++ to see which we would benefit from actively removing and which might now be better undeprecated. Consolidating all this analysis into one place was intended to ease the (L)EWG review process but in return gave the author so much feedback that the next revision of the paper was not completed.

For the C++26 cycle, a much shorter paper, [P2863], will track the overall analysis, but for features that the author wants to actively progress, a distinct paper will decouple progress from the larger paper so that the delays on a single feature do not hold up progress on all.

This paper takes up the deprecated namespace `std::relops` and its comparison operator templates, D.12 [depr.relops].

§ 4 Analysis

NOTE THAT THIS PAPER DOES NOT YET INCORPORATE FEEDBACK FROM THE KONA 2023 MEETING THAT REQUESTED A PAPER TO TRACK THIS PROGRESS SEPARATE FROM [P2863].

While feedback is welcome, we do not anticipate spending more committee time on this until R1 is available.

§ 4.1 History

The `std::rel_ops` namespace was introduced in the original C++ Standard, so its removal would potentially impact on code written against C++98 and later standards. It was deprecated with the introduction of support for the 3-way comparison “spaceship” operator in C++20, by paper [P0768R1].

As of publishing this paper, only the MSVC standard libraries gives deprecation warnings on the sample code in the Tony tables below.

Deprecated	Modern

`#include <cassert>`
`#include <utility>`
 	
`struct Test {`
`  int data = 0;`
 	
`friend bool operator==(Test a, Test b){return a.data == b.data;}`
`friend bool operator <(Test a, Test b){return a.data <  b.data;}`
	`#include <cassert>`
	`#include <compare>`
	
	`struct Test {`
	`  int data = 0;`
	
	`friend bool operator ==(Test, Test) = default;`
	`friend auto operator<=>(Test, Test) = default;`

Deprecated	Modern

`};`
 	
`int main() {`
`  Test x{};`
`  Test y{2};`
 	
`  using namespace std::rel_ops;`
`  assert(x == x);`
`  assert(x != y);`
 	
`  assert(x <  y);`
`  assert(x <= y);`
`  assert(x >= y);`
`  assert(x >  y);`
`}`
	`};`
	
	`int main() {`
	`  Test x{};`
	`  Test y{2};`
	
	`  ~~// No using namespace~~`
	`  assert(x == x);`
	`  assert(x != y);`
	
	`  assert(x <  y);`
	`  assert(x <= y);`
	`  assert(x >= y);`
	`  assert(x >  y);`
	`}`

Tested with MSVC `/W3`, and `-Wall -Wextra -Wdeprecated` with Clang/libc++ and gcc/libstdc++

The orignal version of this paper erroneously claimed that all the comparisons would be synthesized from the same two operations relied upon by the `rel_ops` operators. In fact, those rules rely upon supplying the 3-way comparison operator, rather than `operator<`, as illustrated above.

Note the three changes:

— `#include` a different header

— provide a different operator

  — note that definitions can be defaulted now

— do not rely on a using directive

Given the current lack of deprecation warnings, the tentative recommendation of this paper is to take no action for C++26 and encourage Standard Library maintainers to annotate their implementations as deprecated before the next standard cycle.

As the `using namespace std::rel_ops` idiom may be seen as encouraging bad hygiene, especially when applied at global/namespace scope in a header, there may still be folks motivated to write a paper expressing a stronger intent to remove this feature for C++26.

As a historical note, the Standard Library specification itself used to rely on `std::rel_ops` to provide the specification for any comparison operator if the necessary wording were missing. However, as part of C++20, it was confirmed that no current wording relies on that legacy fall-back, and the corresponding as-if wording was removed.

§ 5 Wording

Make the following changes to the C++ Working Draft. All wording is relative to [N4971], the latest draft at the time of writing.

§ 5.1 Add new identifiers to 16.4.5.3.2 [zombie.names].

§ 16.4.5.3.2 [zombie.names] Zombie names

1 In namespace `std`, the following names are reserved for previous standardization:

- `auto_ptr`,
- `auto_ptr_ref`,
- ...,
- `random_shuffle`,
- `raw_storage_iterator`,
- `relops`,
- `result_of`,
- `result_of_t`,
- ...,
- `undeclare_reachable`, and
- `unexpected_handler`.

§ 5.2 Update Annex C

§ C.1.X Annex D: compatibility features [diff.cpp23.depr]

1 **Change:** Remove namespace `relops` and all its contents.

Rationale: TBD

Effect on original feature: TBD

§ 5.3 Strike wording from Annex D

§ D.12 [depr.relops] Relational operators

1 The header `<utility>` (22.2.1) has the following additions:

```
namespace std::rel_ops {
  template<class T> bool operator!=(const T&, const T&);
  template<class T> bool operator> (const T&, const T&);
  template<class T> bool operator<=(const T&, const T&);
  template<class T> bool operator>=(const T&, const T&);
}
```

To avoid redundant definitions of `operator!=` out of `operator==` and operators `>`, `<=`, and `>=` out of `operator<`, the library provides the following:

```
template<class T> bool operator!=(const T& x, const T& y);
```

Preconditions: `T` meets the *Cpp17EqualityComparable* requirements (Table 28).

Returns: `!(x == y)`.

```
template<class T> bool operator>(const T& x, const T& y);
```

Preconditions: `T` meets the *Cpp17LessThanComparable* requirements (Table 29).

Returns: `y < x`.

```
template<class T> bool operator<=(const T& x, const T& y);
```

Preconditions: `T` meets the *Cpp17LessThanComparable* requirements (Table 29).

Returns: `!(y < x)`.

```
template<class T> bool operator>=(const T& x, const T& y);
```

Preconditions: `T` meets the *Cpp17LessThanComparable* requirements (Table 29).

Returns: `!(x < y)`.

5.4 Update cross-reference for stable labels for C++23

Cross-references from ISO C++ 2023

All clause and subclause labels from ISO C++ 2023 (ISO/IEC 14882:2023, *Programming Languages — C++*) are present in this document, with the exceptions described below.

container.gen.reqmts *see*
container.requirements.general

depr.arith.conv.enum *removed*
depr.codecvt.syn *removed*
depr.default.allocator *removed*
depr.locale.stdcvt *removed*
depr.locale.stdcvt.general *removed*
depr.locale.stdcvt.req *removed*
depr.relops *removed*
depr.res.on.required *removed*
depr.string.capacity *removed*

mismatch *see* alg.mismatch

§ 6 Acknowledgements

Thanks to Michael Park for the pandoc-based framework used to transform this document’s source from Markdown.

§ 7 References

[N4971] Thomas Köppe. 2023-12-18. Working Draft, Programming Languages — C++.
https://wg21.link/n4971

[P0768R1] Walter E. Brown. 2017-11-10. Library Support for the Spaceship (Comparison) Operator.
https://wg21.link/p0768r1

[P2139R2] Alisdair Meredith. 2020-07-15. Reviewing Deprecated Facilities of C++20 for C++23.
https://wg21.link/p2139r2

[P2863] Alisdair Meredith. Review Annex D for C++26.
https://wg21.link/p2863